

This script creates API calls to download the high resolution layer croplands dataset by Copernicus

```
from hda import Client, Configuration
import glob
import os
from pathlib import Path

# initialize a config file with credentials
hdarc = Path(Path.home()/'hdarc')
if not hdarc.is_file():
    import getpass
    USERNAME = input('Enter your username: ')
    PASSWORD = getpass.getpass('Enter your password: ')

    with open(Path.home()/'hdarc', 'w') as f:
        f.write(f'user: {USERNAME}\n')
        f.write(f'password:{PASSWORD}\n')
else:
    print('Configuration file already exists.')

hda_client = Client()
```

Configuration file already exists.

Create the request

Note that currently, only the “Crop Types” dataset is used. The others were analyzed as part of a prototype which was later scrapped.

```

dataset_names = [
    "Crop Types",
#    "Main Crop Emergence",
#    "Main Crop Duration"
]

query = {
    "dataset_id": "EO:EEA:DAT:HRL:CRL",
    "productType": "Crop Types",
    "resolution": "10m",
    "year": "2023",
    "startIndex": 0
}

```

Query The Copernicus server for all available files fitting the parameters

```

for dataset in dataset_names:
    print("processing dataset: ", dataset)
    query["productType"] = dataset

    # query the copernicus server for files matching our filters
    matches = hda_client.search(query)
    print("found matches: ", matches)

    OUTPUT_PATH = os.path.join("../", "data_raw", dataset.replace(" ", "_"))
    print("generated output path: ", OUTPUT_PATH)

    # Check which files are already present in the output directory and remove them from the
    existing_files = glob.glob(os.path.join(OUTPUT_PATH, "**"))
    existing_files = list(set([os.path.basename(e).split(".")[0] for e in existing_files]))
    print(len(existing_files), " files are already in the output directory")
    matches.results = [e for e in matches.results if e["id"] not in existing_files]

    # Download the rest
    print(f"downloading {len(matches.results)} files...")
    matches.download(OUTPUT_PATH)

    # unzip all downloaded files and remove zip archives.
    print("extracting archives...")
    for f in glob.glob(os.path.join(OUTPUT_PATH, "*.zip")):

```

```
new_dir = f.replace(".zip", "")
os.system(f'unzip -n "{f}" -d "{new_dir}"')
os.remove(f)
print("done.")
```

```
processing dataset:  Crop Types
found matches:  SearchResults[items=850,volume=3.4GB]
generated output path:  ../data_raw/Crop_Types
849  files are already in the output directory
downloading 1 files...
```

```
extracting archives...
```

Archive: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00.zip

```
inflating: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00/CLMS_HRL
```

inflating: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00/CLMS_HRL

```
inflating: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00/CLMS_HRL
```

```
inflating: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00/CLMS_HRL
```

```
inflating: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00/CLMS_HRL
```

```
inflating: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00/CLMS_HRL
```

```
inflating: ../data_raw/Crop_Types/CLMS_HRLVLCC_CTY_S2023_R10m_E72N23_03035_V01_R00/CLMS_HRL
```

done.